Supplementary Material for BioRE-KG

Overview

This supplementary document provides implementation details that support the main paper without interrupting its methodological and experimental flow. It contains the exact prompt templates and evidence renderings, experimental configuration and hyperparameters, relation-description strings used as retrieval anchors, and a concise description of the interactive Bio-HybridKG demo application.

1 Prompt Templates and Evidence Rendering

The training and inference workflow uses a stable instruction-following wrapper. The wrapper separates the task instruction, the biomedical input sentence, and the expected response. During supervised fine-tuning, the response field contains the target triple. During inference, generation begins after the response header.

Listing 1: Instruction-following wrapper used for supervised fine-tuning and inference.

```
Below is an instruction that describes a task, paired with an
input that provides further context. Write a response that
appropriately completes the request.

### Instruction:
{instruction}

### Input:
{context}

### Response:
{response}
```

The core extraction instruction defines the output as a serialized biomedical relation triple.

Listing 2: Core biomedical triple-extraction instruction.

```
Please extract the biomedical relation triple from the input
sentence. Use the provided evidence to guide your extraction.
Output format: head_entity|RELATION|tail_entity
```

Retrieved demonstrations are appended to the instruction field as few-shot context–response pairs.

Listing 3: Instruction structure after few-shot exemplar augmentation.

```
{base_task_description}

Examples:
1. Context: {chunk_1} Response: {head_1}|{RELATION_1}|{tail_1}
2. Context: {chunk_2} Response: {head_2}|{RELATION_2}|{tail_2}
...
```

The relation-grounded chunk augmentation stage extends the demonstration block with relation-bearing chunks and alternative evidence arrangements.

Listing 4: Encoding of relation-grounded chunk combinations.

```
{knn_exemplars}|Context: {chunk_a} Response: {RELATION_A}|
Context: {chunk_b} Response: {RELATION_B}|
Context: {chunk_a} Response: {RELATION_A} Context: {chunk_b} ...
```

Relation-grounded chunks are converted into complete triple demonstrations by inserting a compatible head–tail entity pair.

Listing 5: Chunk exemplar before entity replacement.

```
Context: {chunk_text} Response: {RELATION}
```

Listing 6: Chunk exemplar after entity replacement.

```
Context: {head_entity} {chunk_text} {tail_entity}
Response: {head_entity}|{RELATION}|{tail_entity}
```

At inference time, selected structural and textual evidence is appended through a dedicated evidence delimiter.

Listing 7: Template used to append fused evidence to the instruction.

```
{instruction}

--- Knowledge Graph Evidence ---
{fused_context}
```

The fused context is constructed from direct graph edges, multi-hop graph paths, and retrieved textual chunks.

Listing 8: Prompt representation of a direct Knowledge Graph edge.

```
[KG 1-hop | {direction} | src={source}] {subject} --{predicate}--> {object}
```

Listing 9: Prompt representation of a multi-hop Knowledge Graph path.

```
[KG {n}-hop | src={sources}]
  {A} --{R1}--> {B}  ->  {B} --{R2}--> {C}  ->  ...
```

Listing 10: Prompt representation of a retrieved textual chunk.

```
[RAG chunk | sim={cosine_score} | rel={relation}] {chunk_text}
```

The following example shows the final fused evidence block inserted into the inference instruction.

Listing 11: Final fused context block inserted into the inference instruction.

```
=== KG Direct Evidence ===
[KG 1-hop | outgoing | src=DDI] aspirin --INHIBITS--> cox-2
[KG 1-hop | incoming | src=DDI] nsaid --INHIBITS--> cox-2

=== KG Multi-hop Paths ===
[KG 2-hop | src=DDI,MERGED] aspirin --ACTIVATES--> PGE2  ->
           PGE2 --PROMOTES--> pain

=== RAG Text Evidence ===
[RAG chunk | sim=0.924 | rel=INHIBITS] aspirin inhibits cox-2
[RAG chunk | sim=0.891 | rel=INHIBITS] NSAIDs block prostaglandin
```

2 Experimental Configuration and Hyperparameters

Implementation settings were recovered from executable scripts and pinned environment files in the HybridRKG repository. When documentation and code differed, the executable configuration was treated as authoritative. The evaluated backbones were Llama-7B, BioMistral-7B, and GPT-4o. Llama-7B and BioMistral-7B were adapted by supervised fine-tuning with Quantized Low-Rank Adaptation (QLoRA). GPT-4o was accessed through an OpenAI-compatible chat-completions interface and was not fine-tuned.

2.1 Computing and Software Environment

The local experiments were executed in a Linux-based environment using Python 3.8.x, CUDA 11.8, and an NVIDIA GeForce RTX 4090 GPU.

Component	**Llama-7B**	**BioMistral-7B**
Python / CUDA	3.8.x / 11.8	3.8.x / 11.8
PyTorch	2.0.1+cu118	2.0.1+cu118
Transformers	4.31.0	4.46.3
Datasets / Accelerate	2.13.0 / 0.20.3	3.1.0 / 1.0.1
PEFT / TRL	0.4.0 / 0.4.7	0.4.0 / 0.11.4
BitsAndBytes	0.39.1	0.41.3.post2
SentencePiece	0.1.99	0.2.0
NumPy / Pandas	1.24.3 / 2.0.3	1.24.3 / 2.0.3

Table 1: Pinned software environments for the local extraction backbones.

2.2 Relation-Aware Retrieval Configuration

Relation-description and chunk embeddings were produced with a local MedGemma 1.5 4B instruction-tuned checkpoint. It was loaded using 4-bit NormalFloat4 (NF4) quantization, double quantization, bfloat16 computation, float16 model weights, and automatic device mapping. Each representation was the mean of token vectors from the zeroth hidden state.

Retrieval setting	**Value**
Embedding checkpoint	MedGemma 1.5 4B instruction-tuned model
Representation	Mean of zeroth-hidden-state token vectors
Quantization	4-bit NF4 with double quantization
Target chunk length	Approximately 5 words
Development chunks retained	Top 2 per sentence
Initial / retained query candidates	Top 5 / top 1
Permutation cap	50
Embedding batch size	1,000

Table 2: Relation-aware chunk construction and retrieval settings.

2.3 QLoRA Fine-Tuning Configuration

Both local models used the Alpaca-style instruction, input, and response format reported in Section 1. The target was the serialized string `head_entity|RELATION|tail_entity`. Base weights were loaded in 4-bit NF4 mode, and LoRA adapters were applied to the query and value projection layers.

Hyperparameter	Llama-7B	BioMistral-7B
Method	QLoRA SFT	QLoRA SFT
LoRA rank / scaling / dropout	64 / 32 / 0.1	64 / 32 / 0.1
Target modules	`q_proj`, `v_proj`	`q_proj`, `v_proj`
Quantization	4-bit NF4, double quantization	4-bit NF4, double quantization
Batch size / accumulation	1 / 8	1 / 8
Effective single-device batch size	8	8
Learning rate	5×10^{-5}	5×10^{-5}
Maximum steps / save interval	10,000 / 1,000	10,000 / 1,000
Inference checkpoint	8,000	8,000
Maximum gradient norm / warm-up	0.1 / 0.03	0.1 / 0.03
Optimizer	Paged AdamW 8-bit	AdamW (PyTorch)
Training precision	float16	float16
Maximum sequence length	512 tokens	1,024 tokens
Padding	Added `[PAD]`, right	EOS token, right

Table 3: QLoRA supervised fine-tuning configuration.

2.4 Knowledge Graph Retrieval Configuration

Separate directed NetworkX multigraphs were built for GM-CIHT, DDI, and ChemProt, and a merged graph was built from their union. Entity strings were lowercased, relation labels were checked against dataset-specific whitelists, and source sentences and corpus identifiers were stored as provenance.

KG setting	Value
Graph structure	Directed multigraph
Graph variants	GM-CIHT, DDI, ChemProt, and merged
Node normalization	Lowercased entity surface forms
Entity matching	Case-insensitive word-boundary matching
Minimum candidate-node length	4 characters
One-hop direction	Incoming and outgoing edges
Duplicate handling	Unique (h, r, t) tuples retained once
Maximum path depth	2 directed edges
Path constraint	Simple paths
Provenance	Source sentence and corpus identifier

Table 4: Knowledge Graph construction and retrieval settings.

2.5 Evidence Fusion and Experimental Variants

Evidence was ranked using

$$\text{Score}(e, S) = 0.4\,\omega(e) + 0.6\,\text{Sim}_{\text{tf}}(e, S),$$

where $\omega(e) = 1.0$ for direct KG evidence and $\omega(e) = 0.75$ for RAG chunks. Similarity used lowercased alphanumeric tokens, normalized term-frequency vectors, and cosine similarity.

Fusion setting	Value
Source-priority / similarity coefficients	0.4 / 0.6
KG / RAG source priors	1.0 / 0.75
Maximum KG / RAG candidates	20 / 10
General library top-K default	8
Adapter command-line default	2
Evidence quantities evaluated	1, 2, and 3 KG items
Graph-source conditions	Target-dataset KG and merged KG

Table 5: Weighted evidence-fusion settings and evaluated variants.

2.6 Inference Configuration

Llama-7B and BioMistral-7B were loaded in 4-bit NF4 mode with their checkpoint-8,000 LoRA adapters. Input tokenization was limited to 1,500 tokens and generation to 50 new tokens. BioMistral explicitly disabled sampling and used greedy decoding. The Llama script omitted sampling parameters and used the non-sampling default of its Transformers environment.

Setting	Llama-7B	BioMistral-7B	GPT-4o
Adaptation	LoRA checkpoint 8,000	LoRA checkpoint 8,000	None
Maximum input length	1,500	1,500	Service-managed
Maximum new tokens	50	50	Not explicitly set
Sampling	Disabled by default	Explicitly disabled	API-controlled
Temperature	Not explicitly set	Not explicitly set	0.0
Prompt format	Alpaca-style	Alpaca-style	Alpaca-style user message

Table 6: Inference and decoding configuration.

2.7 Reproducibility Qualifications

The training and data-construction scripts do not set Python, NumPy, or PyTorch random seeds. Entity-pair replacement and QLoRA optimization are stochastic, and deterministic CUDA algorithms are not enabled. The repository does not serialize the complete host configuration, such as CPU, system memory, driver version, and per-run resource utilization. Remote GPT-4o outputs may also depend on the provider-side model snapshot even at zero temperature. The settings above therefore support configuration-level reproducibility, but they do not imply bitwise reproducibility.

3 Relation Description Strings

The following natural-language definitions were used as semantic anchors for relation-description embeddings. Capitalization and punctuation are normalized for presentation while preserving the original semantic content.

3.1 GM-CIHT

Table 7: GM-CIHT relation-description strings used as semantic anchors.

Relation	Definition string
PRECEDES	Occurs before or precedes another event.
PREDISPOSES	Increases risk or susceptibility to.
TREATS	Treats or cures a condition.
AFFECTS	Affects or modifies another entity.
INTERACTS_WITH	Directly interacts with another entity.
INHIBITS	Inhibits or blocks another entity.
PRODUCES	Produces or generates another entity.
ADMINISTERED_TO	Is administered to a patient or subject.
PROCESS_OF	Is a process of another entity.
AUGMENTS	Augments or enhances another entity.
PREVENTS	Prevents or stops another condition.
DIAGNOSES	Diagnoses or identifies a condition.
COEXISTS_WITH	Co-occurs or co-exists with another entity.
ASSOCIATED_WITH	Statistically or clinically associated with.
DISRUPTS	Disrupts or interferes with normal function.
CAUSES	Causes or brings about another condition.
COMPLICATES	Complicates or adds complications to.
SYMPTOM_OF	Is a symptom or manifestation of.
DOES_NOT_TREAT	Does not treat or is ineffective against.
STIMULATES	Stimulates or activates another entity.
MANIFESTATION_OF	Is a manifestation of another condition.
USES	Uses or employs another entity.

3.2 DDI

Relation	Definition string
`mechanism`	Pharmacokinetic mechanism of drug–drug interaction.
`effect`	Pharmacodynamic effect or clinical consequence of a drug interaction.
`advise`	Recommendation or advice regarding the concurrent use of two drugs.
`int`	A drug–drug interaction without a detailed mechanism or effect specified.
`None`	No interaction between the two drugs.

Table 8: DDI relation-description strings used as embedding anchors.

3.3 ChemProt

Relation	Definition string
PRODUCT-OF	The chemical is a product of the gene or protein.
DOWNREGULATOR	The chemical downregulates the gene or protein.
ANTAGONIST	The chemical is an antagonist of the gene or protein.
AGONIST	The chemical is an agonist of the gene or protein.
ACTIVATOR	The chemical is an activator of the gene or protein.

Table 9: ChemProt relation-description strings used as embedding anchors.

3.4 Cross-Dataset Semantic Proximity

Several labels are semantically related across datasets but are not automatically merged. `DOWNREGULATOR` may overlap with inhibitory effects expressed by `INHIBITS`; `ACTIVATOR` and some instances of `AGONIST` may overlap with `STIMULATES`; and `PRODUCT-OF` is related to the inverse or domain-specific use of `PRODUCES`. These correspondences are approximate rather than exact because the datasets differ in argument types, directionality, annotation guidance, and granularity. HybridRKG therefore preserves original strings in prompts, targets, and graph edges.

4 Interactive Demo Application

The BioHybridKG demo application was developed as an interactive research artifact for inspecting the inference workflow proposed in the paper. It does not introduce an additional quantitative evaluation protocol. Instead, it makes the pipeline stages visible for qualitative analysis: sentence submission, entity matching, KG and RAG evidence retrieval, evidence fusion, fused-context inspection, and LLM-based triple generation.

4.1 System Architecture

Link to the online web application: `https://huggingface.co/spaces/ntmp1910/HybridRKG_demo`.

The demo follows a lightweight client-server architecture. The frontend is implemented with static web files, including `index.html`, `style.css`, and `app.js`. It provides sidebar controls, example selection, RAG chunk entry, evidence-card rendering, score visualization, KG graph visualization, fused-context inspection, and extracted-triple display. The backend is implemented as a FastAPI service in `app/backend/main.py`. It serves the frontend and exposes API endpoints that connect the interface to the KG retriever, fusion module, and LLM extraction call.

Four backend endpoints are central to the demo workflow. The `/api/kg-stats` endpoint loads statistics for the available KG datasets and displays the merged graph size in the sidebar. The `/api/examples` endpoint returns preset biomedical examples with sentence text, optional ground-truth triples, and example RAG chunks. The `/api/retrieve` endpoint performs the main retrieval and fusion stage. The `/api/extract-triple` endpoint sends the biomedical sentence and fused evidence context to the configured LLM, parses the response in `head|RELATION|tail` format, and compares it with the optional ground truth.

4.2 Interactive Workflow

Figure 1 shows the initial state of the application. The top pipeline indicator presents the three conceptual stages of the demo: KG retrieval, RAG fusion, and LLM extraction.

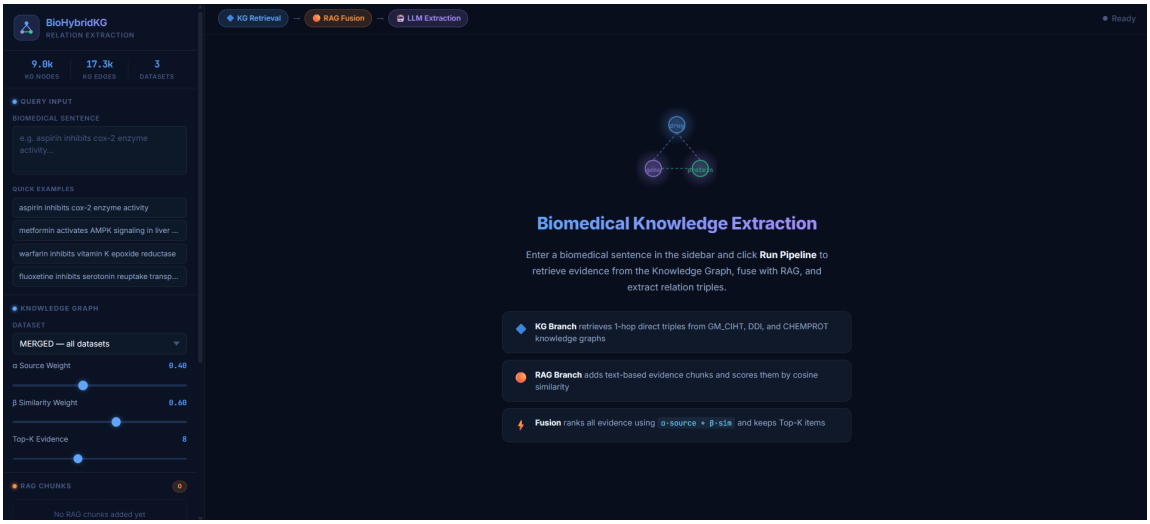

Figure 1: Landing page of the BioHybridKG demo application before a sentence is submitted.

The first interaction step is query preparation. A user may type a biomedical sentence manually or select one of the quick examples. In Figure 2, the selected sentence is `aspirin inhibits cox-2 enzyme activity`. The selected dataset is `MERGED`, which combines the available KG sources.

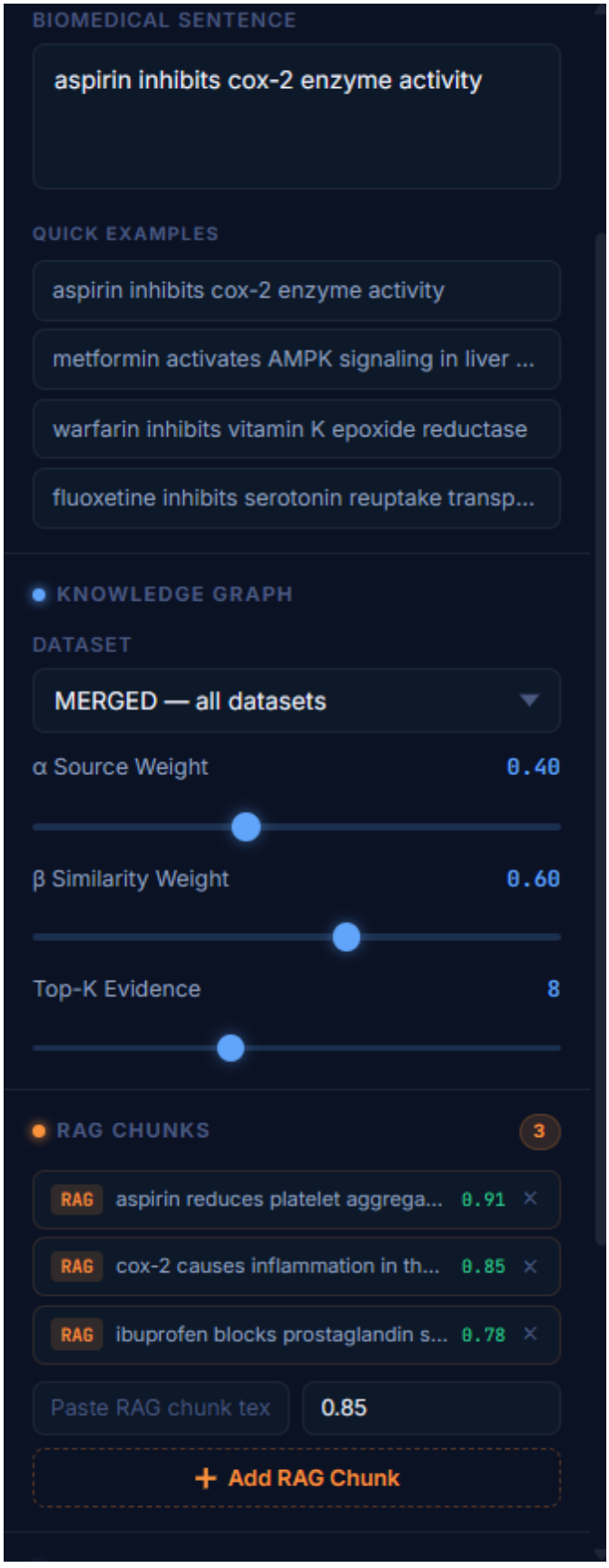

Figure 2: Query setup stage with a selected biomedical sentence, merged KG dataset, fusion weights, top-K setting, and three RAG chunks.

After the user clicks `Run Pipeline`, the frontend sends the sentence, dataset, RAG chunks, fusion weights, and top-K value to `/api/retrieve`. The backend retrieves direct KG evidence for matched entities, formats the RAG chunks, computes the weighted score for each evidence item, and returns the ranked evidence list and fused context.

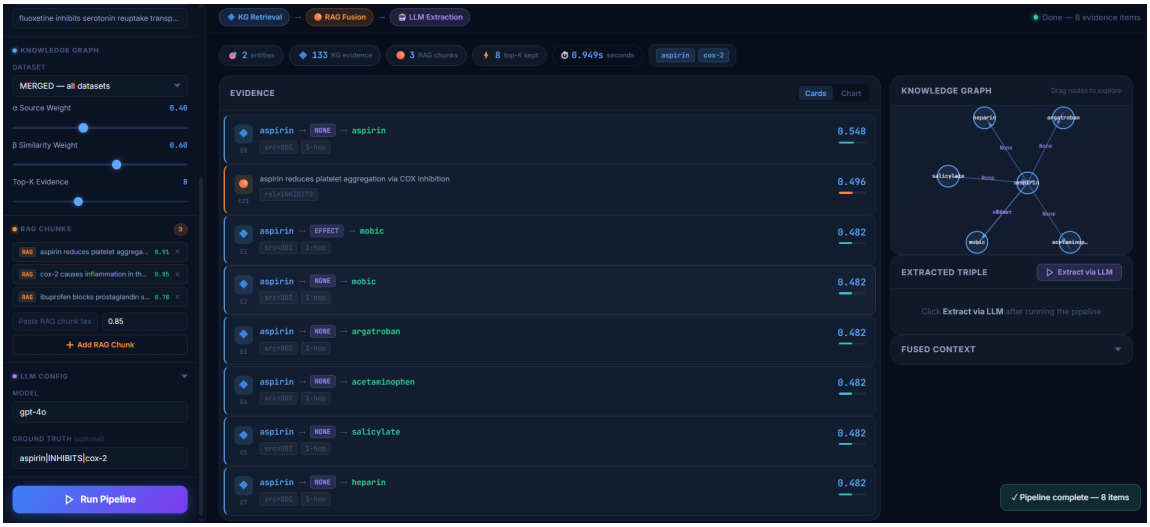

Figure 3: Pipeline result after KG retrieval and RAG fusion, showing ranked evidence cards, matched entities, runtime, and the KG graph visualization.

The extraction stage is triggered only after the retrieval pipeline has completed. The application sends the original sentence and fused KG/RAG context to `/api/extract-triple`. The prompt instructs the LLM to return a single biomedical relation triple in `head_entity|RELATION|tail_entity` format.

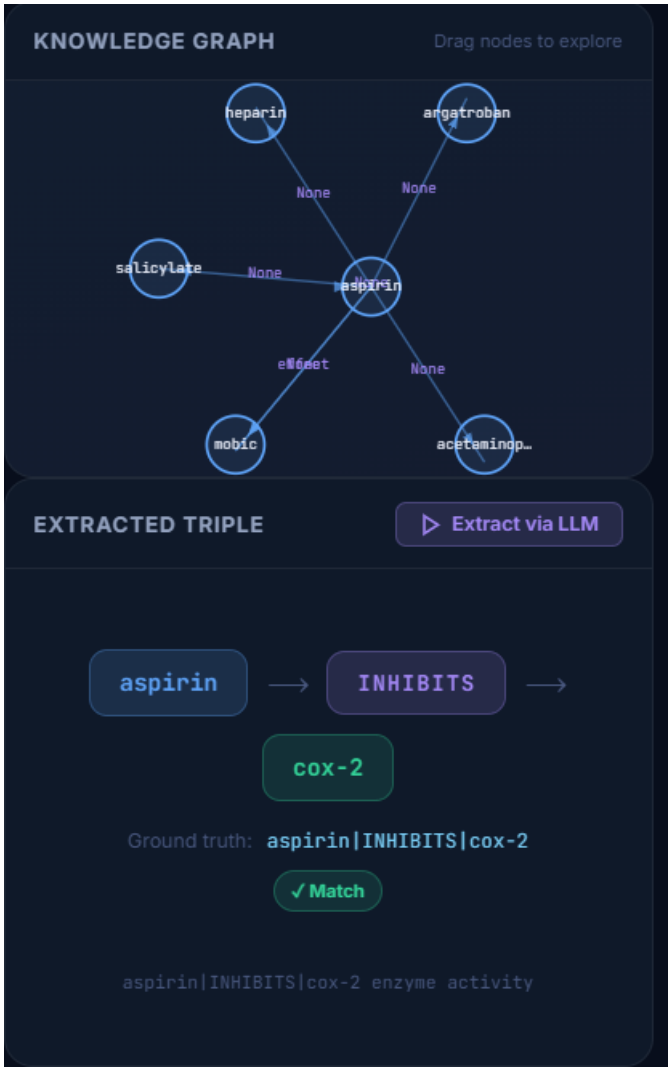

Figure 4: LLM extraction stage after evidence retrieval, where the fused context is used to generate a structured relation triple.

Figure 5 shows the completed run. The evidence list remains visible, the KG graph summarizes structural evidence, and the extracted-triple panel displays the predicted head entity, relation label, and tail entity. When a ground-truth triple is provided, the backend compares it with the LLM output after normalization and displays the match result.

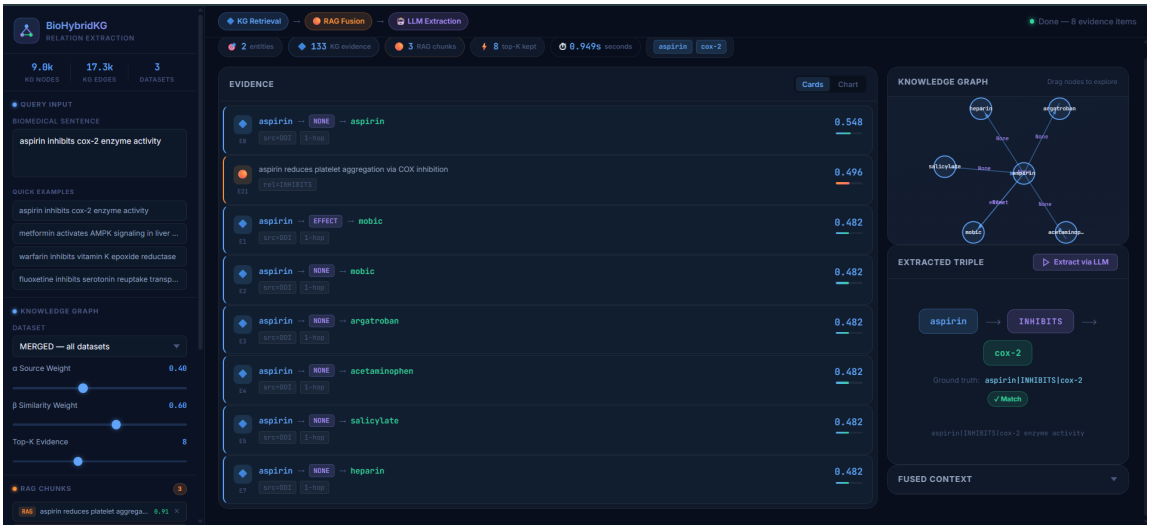

Figure 5: Completed demo run showing ranked evidence, graph visualization, extracted triple, ground-truth triple, and match status.